

Título do Projeto: Solução de Inteligência na Borda com AWS Greengrass V2 para Reconhecimento de Veículos usando YOLO

Nome: Matheus Fortunato Alves

Disciplina: PSI5120

Nº USP: 4484262

Fase 0: Entendendo os Conceitos Fundamentais

Antes de começar, é crucial entender os pilares desta arquitetura.

- **Computação de Borda (Edge Computing):** É um paradigma de computação distribuída que aproxima o processamento de dados da sua fonte de origem (a "borda" da rede). No nosso caso, a borda é o seu ambiente WSL. Em vez de enviar um stream de vídeo bruto para a nuvem para análise, processamos o vídeo localmente e enviamos apenas os resultados (ex: "carro detectado às 15:32:10"), economizando banda, reduzindo a latência e permitindo operação mesmo com conectividade intermitente.
- **AWS Greengrass V2:** É um serviço da AWS que estende os serviços da nuvem para dispositivos de borda. Ele permite que você crie, implante e gerencie software (chamado de "Componentes") em seus dispositivos locais de forma centralizada a partir da nuvem AWS. O "Greengrass Core" é o software principal que roda no seu dispositivo de borda (WSL).
- **Componente Greengrass:** É a unidade de software no Greengrass. Pense nele como um container ou um módulo. Ele é definido por:
 - **Receita (Recipe):** Um arquivo YAML que descreve o componente. Informa o que ele é, suas dependências, de onde baixar os arquivos e como instalá-lo e executá-lo (seu ciclo de vida).
 - **Artefatos (Artifacts):** Os arquivos do seu componente. No nosso caso, serão o script Python, o modelo YOLO pré-treinado (.pt) e um arquivo de dependências (requirements.txt).
- **AWS IoT Core:** É o serviço de nuvem que atua como a porta de entrada para dispositivos IoT. Ele permite que dispositivos (como nosso Greengrass Core) se conectem de forma segura à nuvem AWS e troquem mensagens usando o protocolo MQTT.
- **MQTT (Message Queuing Telemetry Transport):** Um protocolo de mensagens leve, ideal para dispositivos com recursos limitados ou redes instáveis. Ele funciona no modelo "publicar/assinar" (pub/sub). Nosso componente irá *publicar* resultados em um *tópico* (ex: *veiculos/resultados*), e qualquer serviço na nuvem (ou outro dispositivo) pode *assinar* esse tópico para receber os dados.
- **YOLO (You Only Look Once):** Um dos algoritmos de detecção de objetos mais rápidos e populares. Ele "olha" para a imagem uma única vez para detectar múltiplos objetos, suas classes (carro, pessoa, etc.) e suas localizações. Usaremos um modelo pré-treinado para identificar veículos.

Fase 1: Configuração do Ambiente de Borda (WSL)

Aqui, preparamos nosso "dispositivo de borda".

1.1. Habilitar e Instalar o WSL (Subsistema do Windows para Linux)

- Abra o PowerShell ou o Prompt de Comando do Windows **como Administrador**.
- Execute o comando para habilitar os recursos necessários e instalar o Ubuntu (a distribuição padrão):

Bash

```
wsl --install
```

- Após a instalação, reinicie o computador. Ao reiniciar, o Ubuntu será iniciado para que você crie um nome de usuário e uma senha. Guarde essas credenciais.

1.2. Instalar o Docker Engine no Ubuntu (WSL)

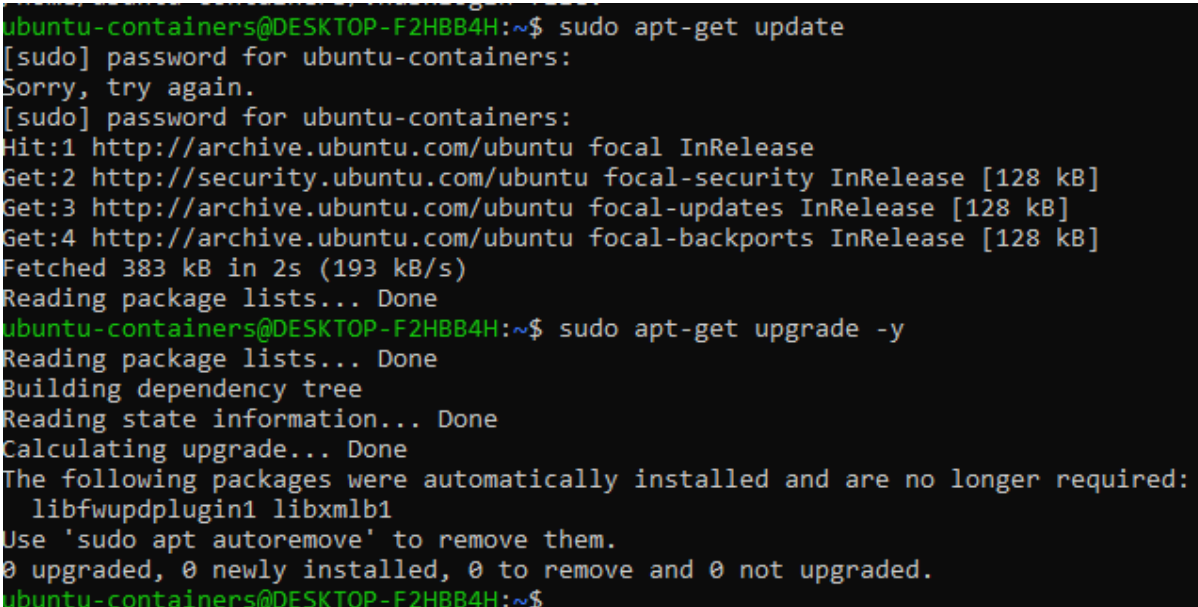
- Abra seu terminal do Ubuntu WSL.

Atualize os pacotes existentes:

Bash

```
sudo apt-get update
```

```
sudo apt-get upgrade -y
```



```
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get update
[sudo] password for ubuntu-containers:
Sorry, try again.
[sudo] password for ubuntu-containers:
Hit:1 http://archive.ubuntu.com/ubuntu focal InRelease
Get:2 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Fetched 383 kB in 2s (193 kB/s)
Reading package lists... Done
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get upgrade -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu-containers@DESKTOP-F2HBB4H:~$
```

Instale os pré-requisitos para o repositório do Docker:

Bash

```
sudo apt-get install -y apt-transport-https ca-certificates curl  
gnupg lsb-release
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get install -y apt-transport-https ca-certificates curl gnupg lsb-release  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
lsb-release is already the newest version (11.1.0ubuntu2).  
lsb-release set to manually installed.  
ca-certificates is already the newest version (20240203~20.04.1).  
ca-certificates set to manually installed.  
curl is already the newest version (7.68.0-1ubuntu2.25).  
gnupg is already the newest version (2.2.19-3ubuntu2.5).  
gnupg set to manually installed.  
The following packages were automatically installed and are no longer required:  
  libfwupdplugin1 libxmlb1  
Use 'sudo apt autoremove' to remove them.  
The following NEW packages will be installed:  
  apt-transport-https  
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.  
Need to get 1704 B of archives.  
After this operation, 162 kB of additional disk space will be used.  
Get:1 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 apt-transport-https all 2.0.11 [1704 B]  
Fetched 1704 B in 0s (4336 B/s)  
Selecting previously unselected package apt-transport-https.  
(Reading database ... 50148 files and directories currently installed.)  
Preparing to unpack .../apt-transport-https_2.0.11_all.deb ...  
Unpacking apt-transport-https (2.0.11) ...  
Setting up apt-transport-https (2.0.11) ...
```

Adicione a chave GPG oficial do Docker:

Bash

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg  
--dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg  
ubuntu-containers@DESKTOP-F2HBB4H:~$
```

Configure o repositório estável do Docker:

Bash

```
echo "deb [arch=$(dpkg --print-architecture)  
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"  
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/  
linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
ubuntu-containers@DESKTOP-F2HBB4H:~$
```

Instale o Docker Engine:

Bash

```
sudo apt-get update  
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
```

```

ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get update
Get:1 https://download.docker.com/linux/ubuntu focal InRelease [57.7 kB]
Get:2 https://download.docker.com/linux/ubuntu focal/stable amd64 Packages [59.9 kB]
Hit:3 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu focal InRelease
Hit:5 http://archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu focal-backports InRelease
Fetched 118 kB in 1s (109 kB/s)
Reading package lists... Done
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get install -y docker-ce docker-ce-cli containerd.io
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfupdpkg1 libxmb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  docker-buildx-plugin docker-ce-rootless-extras docker-compose-plugin pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin pigz slirp4netns
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 120 MB of archives.
After this operation, 439 MB of additional disk space will be used.
Get:1 https://download.docker.com/linux/ubuntu focal/stable amd64 containerd.io amd64 1.7.27-1 [30.5 MB]
Get:2 http://archive.ubuntu.com/ubuntu focal/universe amd64 pigz amd64 2.4-1 [57.4 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal/universe amd64 slirp4netns amd64 0.4.3-1 [74.3 kB]
Get:4 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-buildx-plugin amd64 0.23.0-1~ubuntu.20.04~focal [34.6 MB]
Get:5 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-ce-cli amd64 5:28.1.1-1~ubuntu.20.04~focal [15.8 MB]
Get:6 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-ce amd64 5:28.1.1-1~ubuntu.20.04~focal [19.2 MB]
Get:7 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-ce-rootless-extras amd64 5:28.1.1-1~ubuntu.20.04~focal [6097 kB]
Get:8 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-compose-plugin amd64 2.35.1-1~ubuntu.20.04~focal [13.8 MB]
Fetched 120 MB in 27s (4416 kB/s)
Selecting previously unselected package pigz.
(Reading database ... 50152 files and directories currently installed.)
Preparing to unpack .../0-pigz_2.4-1_amd64.deb ...
Unpacking pigz (2.4-1) ...
Selecting previously unselected package containerd.io.
Preparing to unpack .../1-containerd.io_1.7.27-1_amd64.deb ...
Unpacking containerd.io (1.7.27-1) ...

```

Adicione seu usuário ao grupo do Docker para não precisar usar **sudo** sempre:

Bash

sudo usermod -aG docker \$USER

```

ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo usermod -aG docker $USER
ubuntu-containers@DESKTOP-F2HBB4H:~$

```

- **Importante:** Feche e reabra o terminal do Ubuntu para que essa alteração tenha efeito.

Verifique a instalação:

Bash

docker --version

```

ubuntu-containers@DESKTOP-F2HBB4H:~$ docker --version
Docker version 28.1.1, build 4eba377
ubuntu-containers@DESKTOP-F2HBB4H:~$

```

1.3. Habilite o *systemd* no WSL (necessário para o serviço do Greengrass)

No Ubuntu do WSL, edite `/etc/wsl.conf`:

```
sudo nano /etc/wsl.conf
```

Coloque:

```
[boot]
```

```
systemd=true
```

No Windows, feche os shells e rode:

```
wsl --shutdown
```

Abra o Ubuntu novamente. Verifique:

```
systemctl list-units | head
```

```
ubuntu-containers@DESKTOP-F2H8B4H:~$ systemctl list-units | head
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-0f311f68\x2dea2e\x2d4132\x2da4dc\x2d74cfc604830d-pciea2e:00-ea2e:00:00.0-virtio0-virtio\x2dports-vport0p0.device loaded active plugged /sys/devices/LNXSYSTM:00/LNXSYBUS:00/ACPI0004:00/VMBUS:00/0f311f68-ea2e-4132-a4dc-74cfc604830d/pciea2e:00/ea2e:00:00.0/virtio0/virtio-ports/vport0p0
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-0f311f68\x2dea2e\x2d4132\x2da4dc\x2d74cfc604830d-pciea2e:00-ea2e:00:00.0-virtio0-virtio\x2dports-vport0p1.device loaded active plugged /sys/devices/LNXSYSTM:00/LNXSYBUS:00/ACPI0004:00/VMBUS:00/0f311f68-ea2e-4132-a4dc-74cfc604830d/pciea2e:00/ea2e:00:00.0/virtio0/virtio-ports/vport0p1
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-0f311f68\x2dea2e\x2d4132\x2da4dc\x2d74cfc604830d-pciea2e:00-ea2e:00:00.0-virtio0-virtio\x2dports-vport0p2.device loaded active plugged /sys/devices/LNXSYSTM:00/LNXSYBUS:00/ACPI0004:00/VMBUS:00/0f311f68-ea2e-4132-a4dc-74cfc604830d/pciea2e:00/ea2e:00:00.0/virtio0/virtio-ports/vport0p2
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-d253de02\x2db3d2\x2d4c59\x2d8998\x2d32dd14354d58-net-eth0.device loaded active plugged /sys/devices/LNXSYSTM:00/LNXSYBUS:00/ACPI0004:00/VMBUS:00/d253de02-b3d2-4c59-8998-32dd14354d58/net/eth0
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-target0:0:0:0:0:0-block-sda.device loaded active plugged Virtual_Disk
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-target0:0:0:0:0:0:1-block-sdb.device loaded active plugged Virtual_Disk
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-target0:0:0:0:0:0:2-block-sdc.device loaded active plugged Virtual_Disk
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-target0:0:0:0:0:0:3-block-sdd.device loaded active plugged Virtual_Disk
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-VMBUS:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-target0:0:0:0:0:0:4-block-sde.device loaded active plugged Virtual_Disk
```

1.3. Instalar Pré-requisitos para o Greengrass

Java 11: O Greengrass Core é um software Java.

Bash

`sudo apt-get install -y openjdk-11-jre`

`java -version` # Para verificar

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get install -y openjdk-11-jre
[sudo] password for ubuntu-containers:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  ca-certificates-java fonts-dejavu-extra java-common libatk-wrapper-java libatk-wrapper-java-jni libgif7 openjdk-11-jre-headless
Suggested packages:
  default-jre libnss-mdns fonts-ipafont-gothic fonts-ipafont-mincho fonts-wqy-microhei | fonts-wqy-zenhei fonts-indic
The following NEW packages will be installed:
  ca-certificates-java fonts-dejavu-extra java-common libatk-wrapper-java libatk-wrapper-java-jni libgif7 openjdk-11-jre openjdk-11-jre-headless
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 40.6 MB of archives.
After this operation, 184 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu focal/main amd64 java-common all 0.72 [6816 B]
Get:2 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 openjdk-11-jre-headless amd64 11.0.27+6~us1-0ubuntu1~20.04 [38.3 MB]
Get:3 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 ca-certificates-java all 20190405ubuntu1.1 [12.4 kB]
Get:4 http://archive.ubuntu.com/ubuntu focal/main amd64 fonts-dejavu-extra all 2.37-1 [1953 kB]
Get:5 http://archive.ubuntu.com/ubuntu focal/main amd64 libatk-wrapper-java all 0.37.1-1 [53.0 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal/main amd64 libatk-wrapper-java-jni amd64 0.37.1-1 [45.1 kB]
Get:7 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 libgif7 amd64 5.1.9-1ubuntu0.1 [32.4 kB]
Get:8 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 openjdk-11-jre amd64 11.0.27+6~us1-0ubuntu1~20.04 [194 kB]
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ java -version
openjdk version "11.0.27" 2025-04-15
OpenJDK Runtime Environment (build 11.0.27+6-post-Ubuntu-0ubuntu120.04)
OpenJDK 64-Bit Server VM (build 11.0.27+6-post-Ubuntu-0ubuntu120.04, mixed mode, sharing)
```

Python e Pip: Para o nosso componente de IA.

Bash

`sudo apt-get install -y python3 python3-pip`

```
OpenJDK 64-Bit Server VM (build 11.0.27+6-post-Ubuntu-0ubuntu120.04, mixed mode, sharing)
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get install -y python3 python3-pip
Reading package lists... Done
Building dependency tree
Reading state information... Done
python3 is already the newest version (3.8.2-0ubuntu2).
python3 set to manually installed.
The following packages were automatically installed and are no longer required:
  libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential cpp cpp-9 dpkg-dev fakeroot g++ g++-9 gcc gcc-9 gcc-9-base libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan5 libatomic1 libbinutils libc-dev-bin libc6-dev libc6-i386 libcrypt-dev libctf-nobfd0 libctf0 libdpkg-perl
  libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-9-dev libgomp1 libisl22 libitm1 liblsan0 libmpc3 libpython3-dev libpython3.8-dev libquadmath0
  libstdc++-9-dev libtsan0 libubsan1 linux-libc-dev make manpages-dev python-pip-whl python3-dev python3-wheel python3.8-dev zlib1g-dev
Suggested packages:
  binutils-doc cpp-doc gcc-9-locales debian-keyring g++-multilib g++-9-multilib gcc-9-doc gcc-multilib autoconf automake libtool flex bison gdb gcc-doc gcc-9-multilib
  glibc-doc bzip libstdc++-9-doc make-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential cpp cpp-9 dpkg-dev fakeroot g++ g++-9 gcc gcc-9 gcc-9-base libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan5 libatomic1 libbinutils libc-dev-bin libc6-dev libc6-i386 libcrypt-dev libctf-nobfd0 libctf0 libdpkg-perl
  libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-9-dev libgomp1 libisl22 libitm1 liblsan0 libmpc3 libpython3-dev libpython3.8-dev libquadmath0
  libstdc++-9-dev libtsan0 libubsan1 linux-libc-dev make manpages-dev python-pip-whl python3-dev python3-pip python3-wheel python3.8-dev zlib1g-dev
0 upgraded, 50 newly installed, 0 to remove and 0 not upgraded.
Need to get 52.2 MB of archives.
After this operation, 228 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 binutils-common amd64 2.34-6ubuntu1.11 [208 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 libbinutils amd64 2.34-6ubuntu1.11 [475 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 libctf-nobfd0 amd64 2.34-6ubuntu1.11 [48.2 kB]
```

Outras dependências:

Bash

```
sudo apt-get install -y curl unzip
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo apt-get install -y curl unzip
Reading package lists... Done
Building dependency tree
Reading state information... Done
curl is already the newest version (7.68.0-1ubuntu2.25).
unzip is already the newest version (6.0-25ubuntu1.2).
The following packages were automatically installed and are no longer required:
  libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

1.4. Instalar e Configurar o AWS CLI

O AWS CLI nos permitirá interagir com a AWS pelo terminal.

Bash

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Configurar Credenciais: Você precisará de um usuário IAM na sua conta AWS com permissões programáticas (Chave de Acesso e Chave Secreta).

Bash

```
aws configure
```

- Siga as instruções, inserindo sua **AWS Access Key ID**, **AWS Secret Access Key**, **Default region name** (ex: us-east-1) e **Default output format** (ex: json).

Fase 2: Instalação do AWS Greengrass Core

Agora, vamos transformar nosso WSL em um dispositivo Greengrass gerenciado.

2.1. Criar Recursos na AWS (IAM e IoT)

- O instalador do Greengrass pode criar esses recursos para nós. É a maneira mais fácil de começar. Assegure-se de que o usuário IAM configurado no `aws configure` tenha permissões para criar roles e políticas de IoT (como a `AdministratorAccess` para fins de teste, mas em produção use permissões mais restritas).

2.2. Baixar e Executar o Instalador do Greengrass

No seu terminal Ubuntu WSL, execute o seguinte comando para baixar o software Greengrass Core:

Bash

`curl -s`

`https://d2s8p88vqu9w66.cloudfront.net/releases/greengrass-nucleus-latest.zip > greengrass-nucleus-latest.zip`

`unzip greengrass-nucleus-latest.zip -d GreengrassCore`

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ curl -s https://d2s8p88vqu9w66.cloudfront.net/releases/greengrass-nucleus-latest.zip > greengrass-nucleus-latest.zip
ubuntu-containers@DESKTOP-F2HBB4H:~$ unzip greengrass-nucleus-latest.zip -d GreengrassCore
Archive: greengrass-nucleus-latest.zip
  inflating: GreengrassCore/META-INF/MANIFEST.MF
  inflating: GreengrassCore/META-INF/SIGNER.SF
  inflating: GreengrassCore/META-INF/SIGNER.RSA
  inflating: GreengrassCore/LICENSE
  inflating: GreengrassCore/NOTICE
  inflating: GreengrassCore/README.md
  inflating: GreengrassCore/THIRD-PARTY-LICENSES
  inflating: GreengrassCore/bin/greengrass.exe
  inflating: GreengrassCore/bin/greengrass.service.procd.template
  inflating: GreengrassCore/bin/greengrass.service.template
  inflating: GreengrassCore/bin/greengrass.xml.template
  inflating: GreengrassCore/bin/loader
  inflating: GreengrassCore/bin/loader.cmd
  inflating: GreengrassCore/conf/recipe.yaml
  inflating: GreengrassCore/lib/Greengrass.jar
```

- Agora, execute o instalador. Este comando fará tudo: criar a "Coisa" (Thing) no AWS IoT, criar certificados, políticas, roles e instalar o software como um serviço.
 - **Substitua os valores em `<...>` pelos seus.**

Bash

`sudo -E java -Droot="/greengrass/v2" -Dlog.store=FILE \`

`-jar ./GreengrassCore/lib/Greengrass.jar \`

`--aws-region <sua-regiao-aws> \`

`--thing-name <nome-do-seu-dispositivo-ex:MyWSL-GG-Device> \`

`--thing-group-name <nome-do-grupo-ex:WSL-Devices> \`

`--component-default-user ggc_user \`

`--provision true \`

```
--setup-system-service true \  
  
--deploy-dev-tools true
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo -E java -Droot="/greengrass/v2" -Dlog.store=FILE \  
> -jar ./GreengrassCore/lib/Greengrass.jar \  
--aws-region us-east-1 \  
> --thing-name MyWSL_border_YOLO \  
> --thing-group-name WSL_Devices_Border \  
> --component-default-user ggc_user \  
> --provision true \  
> --setup-system-service true \  
> --deploy-dev-tools true  
Provisioning AWS IoT resources for the device with IoT Thing Name: [MyWSL_border_YOLO]...  
Creating new IoT policy "GreengrassV2IoTThingPolicy"  
Creating keys and certificate...  
Attaching policy to certificate...  
Creating IoT Thing "MyWSL_border_YOLO"...  
Attaching certificate to IoT thing...  
Successfully provisioned AWS IoT resources for the device with IoT Thing Name: [MyWSL_border_YOLO]!  
Adding IoT Thing [MyWSL_border_YOLO] into Thing Group: [WSL_Devices_Border]...  
Successfully added Thing into Thing Group: [WSL_Devices_Border]  
Setting up resources for aws.greengrass.TokenExchangeService ...  
TES role alias "GreengrassV2TokenExchangeRoleAlias" does not exist, creating new alias...  
IoT role policy "GreengrassTESCertificatePolicyGreengrassV2TokenExchangeRoleAlias" for TES Role alias not exist, creating policy...  
Attaching TES role policy to IoT thing...  
No managed IAM policy found, looking for user defined policy...  
IAM policy named "GreengrassV2TokenExchangeRoleAccess" already exists. Please attach it to the IAM role if not already  
Configuring Nucleus with provisioned resource details...  
Downloading CA from "https://www.amazontrust.com/repository/AmazonRootCA1.pem"  
Created device configuration  
Successfully configured Nucleus with provisioned resource details!  
Creating a deployment for Greengrass first party components to the thing group
```

```
> --deploy-dev-tools true  
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo -E java -Droot="/greengrass/v2" -Dlog.store=FILE \  
> -jar ./GreengrassCore/lib/Greengrass.jar \  
> --aws-region us-east-1 \  
> --thing-name MyWSL_border_YOLO \  
> --thing-group-name WSL_Devices_Border \  
> --component-default-user ggc_user \  
> --setup-system-service true \  
> --deploy-dev-tools true  
  
Successfully set up Nucleus as a system service
```

- **Explicando os parâmetros:**

- **Droot="/greengrass/v2"**: Define o diretório raiz da instalação.
- **--aws-region**: A região AWS que você está usando (ex: **us-east-1**).
- **--thing-name**: O nome que seu dispositivo terá no AWS IoT Core.
- **--thing-group-name**: Um grupo para organizar seus dispositivos.
- **--component-default-user ggc_user**: O usuário do sistema que executará os componentes. O instalador o criará para nós.
- **--provision true**: Diz ao instalador para criar os recursos na AWS (Thing, certificados, etc.).
- **--setup-system-service true**: Configura o Greengrass para iniciar automaticamente com o sistema.
- **--deploy-dev-tools true**: Implanta um componente de CLI útil para depuração local.

2.3. Verificar a Instalação

Após a execução do script, o Greengrass Core deve estar rodando. Verifique o status:

Bash

```
sudo systemctl status greengrass.service
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo systemctl status greengrass.service
* greengrass.service - Greengrass Core
   Loaded: loaded (/etc/systemd/system/greengrass.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2025-08-30 16:04:18 -03; 36s ago
     Main PID: 1806 (loader)
       Tasks: 49 (limit: 9464)
      Memory: 256.6M
         CPU: 17.718s
    CGroup: /system.slice/greengrass.service
            └─1806 /bin/sh -x /greengrass/v2/alts/current/distro/bin/loader
              └─1815 java -Dlog.store=FILE -Dlog.store=FILE -Droot=/greengrass/v2 -jar /greengrass/v2/alts/current/distro/lib/Greengrass.jar --setup-system-service false

Aug 30 16:04:18 DESKTOP-F2HBB4H systemd[1]: Started Greengrass Core.
```

- Você deve ver **active (running)**.

Verifique os logs para garantir que tudo está se comunicando com a nuvem:

Bash

```
sudo tail -f /greengrass/v2/logs/greengrass.log
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~$ sudo tail -f /greengrass/v2/logs/greengrass.log
2025-08-30T19:04:58.470Z [INFO] (pool-3-thread-6) com.aws.greengrass.deployment.DeploymentService: deployment-task-execution. Finished deployment task. {deploymentId=0ca76197-d962-4f7d-af67-1b95a3cc0ad7, serviceName=DeploymentService, currentState=RUNNING}
2025-08-30T19:04:58.470Z [INFO] (pool-3-thread-7) com.aws.greengrass.lifecyclemanager.UpdateSystemPolicyService: service-update-finish. {serviceName=UpdateSystemPolicyService, currentState=RUNNING}
2025-08-30T19:05:08.198Z [INFO] (pool-3-thread-10) com.aws.greengrass.deployment.DeploymentService: Current deployment finished. {DeploymentId=be00356f-7198-462d-9a54-f15466db968c, GreengrassDeploymentId=0ca76197-d962-4f7d-af67-1b95a3cc0ad7, serviceName=DeploymentService, currentState=RUNNING}
2025-08-30T19:05:08.623Z [INFO] (pool-3-thread-10) com.aws.greengrass.deployment.DeploymentStatusKeeper: Stored deployment status. {DeploymentId=be00356f-7198-462d-9a54-f15466db968c, GreengrassDeploymentId=0ca76197-d962-4f7d-af67-1b95a3cc0ad7, DeploymentStatus=SUCCEEDED}
2025-08-30T19:05:08.624Z [INFO] (pool-3-thread-10) com.aws.greengrass.deployment.IotJobsHelper: Updating status of persisted deployment. {Status=SUCCEEDED, StatusDetails={detailed-deployment-status=SUCCESSFUL, ThingName=MyWSL_border_YOLO, JobId=be00356f-7198-462d-9a54-f15466db968c}}
2025-08-30T19:05:09.161Z [INFO] (AwsEventLoop 3) com.aws.greengrass.deployment.IotJobsHelper: Job status update was accepted. {Status=SUCCEEDED, ThingName=MyWSL_border_YOLO, JobId=be00356f-7198-462d-9a54-f15466db968c}
2025-08-30T19:05:09.367Z [INFO] (AwsEventLoop 3) com.aws.greengrass.deployment.IotJobsHelper: Received empty jobs in notification. {ThingName=MyWSL_border_YOLO}
2025-08-30T19:05:09.367Z [INFO] (AwsEventLoop 3) com.aws.greengrass.deployment.IotJobsHelper: Added cancellation deployment to the queue. {ThingName=MyWSL_border_YOLO}
2025-08-30T19:05:09.517Z [INFO] (pool-3-thread-10) com.aws.greengrass.status.FleetStatusService: fss-status-update-published. Status update published to FSS. {trigger=THING_GROUP_DEPLOYMENT, serviceName=FleetStatusService, currentState=RUNNING}
2025-08-30T19:05:09.518Z [INFO] (pool-3-thread-10) com.aws.greengrass.deployment.DeploymentDirectoryManager: Persist link to last deployment. {link=/greengrass/v2/deployments/previous-success}
```

- Procure por mensagens de conexão bem-sucedida com o IoT Core.

Vá até o console da AWS -> IoT Core -> Greengrass -> Core devices. Você deve ver seu **Greengrass device**

Dispositivos de núcleo do Greengrass (1)

Um dispositivo de núcleo do AWS IoT Greengrass é um dispositivo que executa o software Greengrass Core. Depois que seu dispositivo for provisionado, ele aparecerá aqui.

Q

Encontrar dispositivos de núcleo

Nome

Status

Tempo de execução

Status relatado

MyWSL_border_YOLO

Integridade

aws_nucleus_classic

há 3 minutos

Configurar a descoberta na nuvem

Configurar o dispositivo de núcleo

<

1

>

Fase 3: Desenvolvimento do Componente de Reconhecimento de Veículos

3.1 Escolha do modelo YOLO

Usaremos **Ultralytics YOLOv8n** (leve, bom para CPU). Baixaremos os pesos e chamaremos a API Python `YOLO(weights).predict(...)`. Classes de interesse: `car`, `truck`, `bus`, `motorbike`. docs.ultralytics.com+1

Conceito: **Inferência na borda** reduz latência e evita subir vídeos para a nuvem; mandamos **somente metadados** (classe, *confidence*, *bbox*).

3.2 Estrutura de artefatos

Vamos criar uma pasta de artefatos que depois iremos **enviar para o S3**, pois a **recipe** referência URIs `s3://...`:

`artifacts/`

```
└─ vehicle-detector/1.0.0/
    │
    │   └─ main.py
    │   └─ requirements.txt
    │   └─ models/
    │       │   └─ yolov8n.pt
    └─ config.yaml    (opcional)
```

requirements.txt

```
ultralytics
opencv-python-headless
numpy
awscli
awscli
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts$ mkdir vehicle-detector
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts$ cd vehicle-detector
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector$ mkdir 1.0.0
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector$ cd 1.0.0
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0$ nano requirements.txt
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0$
```

```
GNU nano 4.8
ultralytics
opencv-python-headless
numpy
awscli
awscli
```

A lib **awscli** fornece o cliente **IPC** para publicar em IoT Core **sem precisar gerenciar certificados no seu código** (o Nucleus faz isso via Token Exchange). A operação usada será **PublishToIoTCore**. [Documentação AWS](#)

3.3 Baixe o peso do YOLO e salve em **models/yolov8n.pt**

No WSL:

```
mkdir -p artifacts/vehicle-detector/1.0.0/models
```

```
python3 - <<'PY'
```

```
from ultralytics import YOLO
```

```
m = YOLO('yolov8n.pt')
```

```
m.export(format='pt') # força baixar se não tiver
```

```
print("Baixado.")
```

```
PY
```

```
# Copie o arquivo baixado (geralmente ~/.cache/ultralytics/ ou ./)
para artifacts/vehicle-detector/1.0.0/models/yolov8n.pt
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0/models$ python3 - <<'PY'
> from ultralytics import YOLO
> m = YOLO('yolov8n.pt')
> m.export(format='pt') # força baixar se não tiver
> print("Baixado.")
py> PY
WARNING ⚠ user config directory '/home/ubuntu-containers/.config/ultralytics' is not writeable, defaulting to '/tmp' or CWD. Alternatively you can define a YOLO_CONFIG
DIR environment variable for this path.
Creating new Ultralytics Settings v0.0.6 file @
View Ultralytics Settings with 'yolo settings' or at '/tmp/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt to 'yolov8n.pt': 100% _____ 6.2/6.2MB 4.8MB/s 1.3s
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
  File "/home/ubuntu-containers/.local/lib/python3.8/site-packages/ultralytics/engine/model.py", line 734, in export
    return Exporter(overrides=args, _callbacks=self.callbacks)(model=self.model)
  File "/home/ubuntu-containers/.local/lib/python3.8/site-packages/ultralytics/engine/exporter.py", line 284, in __call__
    raise ValueError(f"Invalid export format='{fmt}'. Valid formats are {fmts}")
ValueError: Invalid export format='pt'. Valid formats are ('torchscript', 'onnx', 'openvino', 'engine', 'coreml', 'saved_model', 'pb', 'tflite', 'edgetpu', 'tfjs', 'pad
le', 'mnn', 'ncnn', 'imx', 'rknn')
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0/models$ ls
yolov8n.pt
```

3.4 main.py (com explicações embutidas)

```
#!/usr/bin/env python3

import argparse

import json

import time

import os

import sys

import logging

from datetime import datetime


import cv2

from ultralytics import YOLO


# SDK do Greengrass para IPC

from awsiot.greengrasscoreipc.clientv2 import GreengrassCoreIPCClientV2

from awsiot.greengrasscoreipc.model import QOS, ServiceError


# --- Configuração de logging ---

logging.basicConfig(stream=sys.stdout, level=logging.INFO)

logger = logging.getLogger(__name__)


# --- Função para publicar detecções ---

def publish_detection(ipc_client: GreengrassCoreIPCClientV2, topic: str, payload: dict):

    try:

        payload_json = json.dumps(payload).encode("utf-8")

        ipc_client.publish_to_iot_core(

            topic_name=topic,

            qos=QOS.AT_LEAST_ONCE,

            payload=payload_json

        )
```

```

        logger.info(f"Mensagem publicada com sucesso no tópic: {topic}")

except ServiceError as e:

    logger.error(f"Erro ao publicar no IoT Core: {e}", exc_info=True)

except Exception as e:

    logger.error(f"Erro inesperado durante a publicação: {e}", exc_info=True)


# --- Função principal ---

def main():

    parser = argparse.ArgumentParser(description="Detector de Veículos YOLOv8 para AWS Greengrass")

    parser.add_argument("--source", required=True, help="Arquivo de vídeo")

    parser.add_argument("--topic", required=True, help="Tópico MQTT para publicar detecções")

    parser.add_argument("--conf", type=float, default=0.50, help="Confiança mínima para detecção")

    parser.add_argument("--labels", required=True, help="Classes a serem detectadas, separadas por vírgula")

    parser.add_argument("--model", default=os.path.join(os.path.dirname(__file__), "models", "yolov8n.pt"), help="Caminho do modelo YOLOv8")


    args = parser.parse_args()


    # --- Configura cache gravável para gravar resultados---

    cache_dir = "/greengrass/v2/work/com.example.VehicleDetector/cache"

    os.makedirs(cache_dir, exist_ok=True)

    os.environ["ULTRALYTICS_CACHE"] = cache_dir


    logger.info("---- Iniciando VehicleDetector ----")

    logger.info(f"Fonte do vídeo: {args.source}")

    logger.info(f"Tópico MQTT: {args.topic}")

    logger.info(f"Confiança mínima: {args.conf}")

    logger.info(f"Modelo YOLO: {args.model}")

    logger.info(f"Cache dir: {cache_dir}")

```

```
allowed_labels = {s.strip() for s in args.labels.split(",") if s.strip()}

logger.info(f"Classes permitidas: {allowed_labels}")

# --- Carrega modelo YOLO ---

try:
    model = YOLO(args.model)
except Exception as e:
    logger.error(f"Falha ao carregar modelo YOLO: {e}", exc_info=True)
    sys.exit(1)

# --- Inicializa captura de vídeo ---

cap_source = 0 if args.source == "0" else args.source
cap = cv2.VideoCapture(cap_source)

if not cap.isOpened():
    logger.error(f"Não foi possível abrir a fonte de vídeo: '{args.source}'")
    sys.exit(1)

# --- Inicializa IPC do Greengrass ---

ipc_client = None

for i in range(5):
    try:
        ipc_client = GreengrassCoreIPCClientV2()

        logger.info("Cliente IPC conectado com sucesso ao Nucleus do Greengrass.")

        break
    except Exception as e:
        logger.warning(f"Falha ao conectar IPC, retry em 5s ({i+1}/5)...")
        time.sleep(5)

if ipc_client is None:
    logger.error("Não foi possível conectar ao Greengrass IPC")
    sys.exit(1)
```

```
# --- Loop principal ---

try:

    while True:

        ok, frame = cap.read()

        if not ok:

            logger.info("Fim do vídeo ou fonte indisponível. Reiniciando em 10s.")

            cap.release()

            time.sleep(10)

            cap = cv2.VideoCapture(cap_source)

            if not cap.isOpened():

                logger.error("Falha ao reabrir a fonte de vídeo. Encerrando.")

                break

            continue

# --- Inferência ---

t_start = time.time()

results = model.predict(source=frame, verbose=False, conf=args.conf, imgsz=640)[0]

t_latency = int((time.time() - t_start) * 1000)

# --- Processa detecções ---

detections = []

for box in results.boxes:

    class_id = int(box.cls.item())

    class_name = results.names[class_id]

    if class_name in allowed_labels:

        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())

        detections.append({

            "class": class_name,

            "confidence": round(float(box.conf.item()), 3),

            "bbox": [x1, y1, x2, y2]

        })
```

```

# --- Publica resultados ---

message_payload = {

    "timestamp": datetime.utcnow().isoformat() + "Z",

    "source": args.source,

    "inference_latency_ms": t_latency,

    "detection_count": len(detections),

    "detections": detections

}

publish_detection(ipc_client, args.topic, message_payload)

except KeyboardInterrupt:

    logger.info("Recebido sinal de interrupção. Encerrando...")

except Exception as e:

    logger.error(f"Erro inesperado no loop principal: {e}", exc_info=True)

finally:

    cap.release()

    logger.info("--- VehicleDetector finalizado ---")

if __name__ == "__main__":

    main()

```

Pontos importantes:

- Usamos **Greengrass IPC PublishToIoTCore** (QoS 1) — nada de gerenciar certificados manualmente. [Documentação AWS](#)
- Filtramos classes de **veículos** para reduzir payload.
- Enviamos **metadados** (timestamp, latência, bboxes).
- O script aceita **arquivo de vídeo** (ex.: `video.mp4`)
- **Ultralytics YOLOv8**: instalação e uso seguem a doc oficial. docs.ultralytics.com+1

Fase 4: Publicação e Implantação do Componente

4.1 Crie um bucket S3 (na mesma região do seu Greengrass)

```
AWS_REGION=us-east-1    # ajuste
```

```
BUCKET=gg-artifacts-$RANDOM
```

```
aws s3 mb s3://$BUCKET --region $AWS_REGION
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0$ AWS_REGION=us-east-1
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0$ BUCKET=gg-artifacts-$RANDOM
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0$ aws s3 mb s3://$BUCKET --region $AWS_REGION
make_bucket: gg-artifacts-22274
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector/artifacts/vehicle-detector/1.0.0$
```

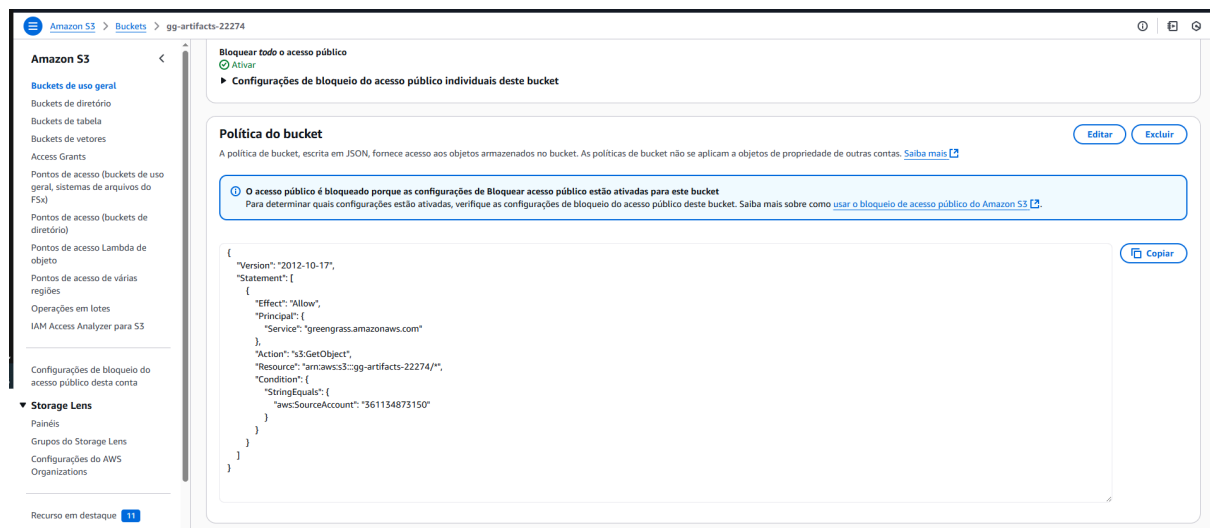
4.2 Envie seus arquivos

```
aws s3 cp --recursive artifacts/vehicle-detector/1.0.0 \
s3://$BUCKET/components/vehicle-detector/1.0.0/
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$ aws s3 cp --recursive artifacts/vehicle-detector/1.0.0 \
> s3://$BUCKET/components/vehicle-detector/1.0.0/
upload: artifacts/vehicle-detector/1.0.0/main.py to s3://gg-artifacts-22274/components/vehicle-detector/1.0.0/main.py
upload: artifacts/vehicle-detector/1.0.0/requirements.txt to s3://gg-artifacts-22274/components/vehicle-detector/1.0.0/requirements.txt
upload: artifacts/vehicle-detector/1.0.0/models/yolov8n.pt to s3://gg-artifacts-22274/components/vehicle-detector/1.0.0/models/yolov8n.pt
upload: artifacts/vehicle-detector/1.0.0/road_video.mp4 to s3://gg-artifacts-22274/components/vehicle-detector/1.0.0/road_video.mp4
```

4.3 Adicione política permissão de leitura no bucket

Na página do AWS Console S3, vá para Buckets → Selecione seu bucket. Dentro do bucket, na aba permissões, adicione as seguintes políticas



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "greengrass.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::gg-artifacts-22274/*",
      "Condition": {
        "StringEquals": {
          "aws:SourceAccount": "361134873150"
        }
      }
    }
  ]
}
```

Fase 5: Crie a Recipe do componente

A **recipe** descreve:

- Nome/versão do componente
- Artefatos (nossos arquivos **s3://...**)
- *Lifecycle* (como instalar e como executar)
- **Access control** (política de autorização IPC para publicar em IoT Core)

Crie `vehicle-detector-1.0.0.yaml`:

— — —

RecipeFormatVersion: '2020-01-25'

```
ComponentName: com.example.VehicleDetector
```

```
ComponentVersion: '1.0.0'
```

ComponentDescription: Detector de veículos YOLOv8 para AWS Greengrass V2

ComponentPublisher: SeuNome

ComponentDependencies:

aws.greengrass.Nucleus:

```
VersionRequirement: '>=2.0.0'
```

DependencyType: HARD

```
# Configuração padrão do componente com accessControl
```

```
# A fonte de dados é um vídeo gravado de uma rodovia com a passagem de muitos veículos
(road_video.mp4), mas poderia ser um vídeo/imagem enviada por uma câmera de segurança em tempo
real para o AWS.
```

A confiança mínima para considerar uma detecção das classes configuradas é de 0.5 (configurável). As classes avaliadas pelo script são "car, truck, bus, motorbike".

```
# Os resultados são publicados no tópico MQTT edge/vehicles/detections
```

ComponentConfiguration:

DefaultConfiguration:

```
source: "road_video.mp4"
```

```
topic: "edge/vehicles/detections"
```

confidence: 0.5

```
labels: "car,truck,bus,motorbike"
```

accessControl:

```
aws.greengrass.ipc.mqttproxy:
```

```
com.example.VehicleDetector:mqtt:1:
```

```
policyDescription: "Permite publicar no tópico de resultados"
```

```
operations:
```

```
- "aws.greengrass#PublishToIoTCore"
```

```
resources:

  - "edge/vehicles/*"

# Manifestos de plataforma

Manifests:

- Platform:

  os: linux

  Artifacts: #Aponta para o diretório com os arquivos no Greengrass device.

  - URI: "s3://gg-artifacts-22274/components/vehicle-detector/1.0.0/main.py"

  - URI: "s3://gg-artifacts-22274/components/vehicle-detector/1.0.0/models/yolov8n.pt"

  - URI: "s3://gg-artifacts-22274/components/vehicle-detector/1.0.0/road_video.mp4"

Lifecycle:

  Install:

    RequiresPrivilege: true

    Script: |

      set -ex

      python3 -m venv {work:path}/venv

      . {work:path}/venv/bin/activate

      pip install --upgrade pip

      pip install -r {artifacts:path}/requirements.txt

      mkdir -p {work:path}/cache

  Run:

    RequiresPrivilege: true

    Script: |

      set -ex

      . {work:path}/venv/bin/activate

      python3 {artifacts:path}/main.py \

        --source "{artifacts:path}/road_video.mp4" \

        --model "{artifacts:path}/models/yolov8n.pt" \

        --topic "{configuration:/topic}" \
```

```
--conf {configuration:/confidence} \  
  
--labels "{configuration:/labels}"
```

Fase 6: Registre a receita e crie a versão do componente na nuvem

No WSL (substitua `{{BUCKET}}` na receita pelo nome real do seu bucket antes):

```
aws greengrassv2 create-component-version \  
  
--inline-recipe fileb://vehicle-detector-1.0.0.yaml
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$ aws greengrassv2 create-component-version --inline-recipe fileb://vehicle-detector-1.0.0.yaml  
{  
  "arn": "arn:aws:greengrass:us-east-1:361134873150:components:com.example.VehicleDetector:versions:1.0.0",  
  "componentName": "com.example.VehicleDetector",  
  "componentVersion": "1.0.0",  
  "creationTimestamp": "2025-08-30T21:56:38.714000-03:00",  
  "status": {  
    "componentState": "REQUESTED",  
    "message": "NONE",  
    "errors": {},  
    "vendorGuidance": "ACTIVE",  
    "vendorGuidanceMessage": "NONE"  
  }  
}
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$ aws s3 ls s3://gg-artifacts-4604/components/vehicle-detector/1.0.0/  
PRE models/  
2025-08-30 18:20:15      3839 main.py  
2025-08-30 18:20:14      166 requirements.txt
```

Esse comando cria **componente** na nuvem. Você pode listar depois com `aws greengrassv2 list-components`. [AWS CLI+1](#)

Fase 7: Faça o *deploy* do componente para o seu Core Device

7.1 Primeiro, descubra o nome do Core Device (Thing) criado na instalação (fica visível no console do IoT Core em Greengrass > Core devices). No CLI:

```
aws greengrassv2 list-core-devices
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$ aws greengrassv2 list-core-devices --region us-east-1
{
  "coreDevices": [
    {
      "coreDeviceThingName": "MyWSL_border_YOLO",
      "status": "HEALTHY",
      "lastStatusUpdateTimestamp": "2025-08-30T17:39:39.910000-03:00",
      "platform": "linux",
      "architecture": "amd64",
      "runtime": "aws_nucleus_classic"
    }
  ]
}
```

7.2 Crie o deployment:

```
aws greengrassv2 create-deployment \

--target-arn arn:aws:iot:AWS_REGION:Account_ID:thing/THING_NAME \

--deployment-name "deploy-name" \

--components '{

    "aws.greengrass.Nucleus": { "componentVersion": "2.12.2" },

    "com.example.VehicleDetector": { "componentVersion": "1.0.0" }

}'
```

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$ aws greengrassv2 create-deployment \
--target-arn arn:aws:iot:us-east-1:361134873150:thing/MyWSL_border_YOLO \
--deployment-name "VICTORY-LAP" \
--components '{
  "aws.greengrass.Nucleus": { "componentVersion": "2.12.2" },
  "com.example.VehicleDetector": { "componentVersion": "1.0.0" }
}'
{
  "deploymentId": "b64d2ee4-c0b7-4f84-b6f4-3b57e40d6507"
}
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$
```

7.3 Acompanhe no host:

`sudo /greengrass/v2/bin/greengrass-cli component list`

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$ sudo /greengrass/v2/bin/greengrass-cli component list
[sudo] password for ubuntu-containers:
Components currently running in Greengrass:
Component Name: TelemetryAgent
  Version: 0.0.0
  State: RUNNING
  Configuration: null
Component Name: aws.greengrass.Nucleus
  Version: 2.12.2
  State: FINISHED
  Configuration: {"awsRegion":"us-east-1","componentStoreMaxSizeBytes":"1000000000","deploymentConfigurationTimeSource":"deploymentCreationTime","deploymentPollingFrequencySeconds":"15","envStage":"prod","fipsMode":"false","fleetStatus":{"periodicStatusPublishIntervalSeconds":86400.0},"greengrassDataPlaneEndpoint":"","greengrassDataPlanePort":"8443","httpClient":{"interpolateComponentConfiguration":false,"iotCredEndpoint":"c3fpk1fcb4cfqx.credentials.iot.us-east-1.amazonaws.com","iotDataEndpoint":"","a7195uafj0050-ats.iot.us-east-1.amazonaws.com","iotRoleAlias":"GreengrassV2TokenExchangeRoleAlias"},"jvmOptions":"","Dlog.store":"FILE","logging":{"mqtt":{"spooler":{"networkProxy":{"proxy":{"platformOverride":{"runWithDefault":{"posixShell":"sh","posixUser":"ggc_user"},"s3EndpointType":"GLOBAL","telemetry":{"Component Name: FleetStatusService
  Version: 0.0.0
  State: RUNNING
  Configuration: null
Component Name: DeploymentService
  Version: 0.0.0
  State: RUNNING
  Configuration: null
Component Name: UpdateSystemPolicyService
  Version: 0.0.0
  State: RUNNING
  Configuration: null
Component Name: com.example.VehicleDetector
  Version: 1.0.0
  State: RUNNING
  Configuration: {"accessControl":{"aws.greengrass.ipc.mqttproxy":{"com.example.VehicleDetector:mqtt:1":{"operations":["aws.greengrass#PublishToIoTCore"],"policyDescription":"Permite publicar no t\u00f3pico de resultados","resources":["edge/vehicles/*"]}}},"confidence":0.5,"labels":{"car,truck,bus,motorbike"},"source":"road_video.mp4","topic":"edge/vehicles/detections"}
Component Name: aws.greengrass.CLI
  Version: 2.15.0
  State: RUNNING
  Configuration: {"AuthorizedPosixGroups":null,"AuthorizedWindowsGroups":null}
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$
```

`sudo tail -f /greengrass/v2/logs/greengrass.log`

```
ubuntu-containers@DESKTOP-F2HBB4H:~/yolo_vehicle_detector$ sudo tail -f /greengrass/v2/logs/greengrass.log
2025-08-30T21:54:55.309Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.RpcServer: New connection code [AWS_ERROR_SUCCESS] for [Id 35, Class ServerConnection, Refs 1](2025-08-30T21:54:55.309555Z) - <null>. {}
2025-08-30T21:54:55.364Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.ServiceOperationMappingContinuationHandler: aws.greengrass#GreengrassCoreIPC authenticated identity: greengrass-cli#user-0. {}
2025-08-30T21:54:55.364Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.ServiceOperationMappingContinuationHandler: Connection accepted for greengrass-cli#user-0. {}
2025-08-30T21:54:55.365Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.ServiceOperationMappingContinuationHandler: Sending connect response for greengrass-cli#user-0. {}
2025-08-30T21:54:55.448Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.RpcServer: Server connection closed code [socket is closed.]: [Id 35, Class ServerConnection, Refs 1](2025-08-30T21:54:55.309555Z) - <null>. {}
2025-08-30T21:55:15.652Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.RpcServer: New connection code [AWS_ERROR_SUCCESS] for [Id 37, Class ServerConnection, Refs 1](2025-08-30T21:55:15.652219Z) - <null>. {}
2025-08-30T21:55:15.713Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.ServiceOperationMappingContinuationHandler: aws.greengrass#GreengrassCoreIPC authenticated identity: greengrass-cli#user-0. {}
2025-08-30T21:55:15.713Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.ServiceOperationMappingContinuationHandler: Connection accepted for greengrass-cli#user-0. {}
2025-08-30T21:55:15.713Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.ServiceOperationMappingContinuationHandler: Sending connect response for greengrass-cli#user-0. {}
2025-08-30T21:55:15.787Z [INFO] (AwsEventLoop 1) software.amazon.awssdk.eventstreamrpc.RpcServer: Server connection closed code [socket is closed.]: [Id 37, Class ServerConnection, Refs 1](2025-08-30T21:55:15.652219Z) - <null>. {}
```

O CLI local ajuda a inspecionar componentes, reiniciar, ver status etc.

[Documenta\u00e7\u00e3o AWS](#)

Fase 8: Valida\u00e7\u00e3o: ver as mensagens no MQTT (AWS IoT Core)

8.1 No console do AWS IoT Core, abra MQTT test client e assine o t\u00f3pico:

- T\u00f3pico: `edge/vehicles/#` (coringa para ver todos)

Você deverá ver mensagens JSON com `{ ts, latency_ms, count, detections: [...] }`.

[illegible]

O cliente de teste do IoT Core permite **subscribe/publish** direto no navegador e é ótimo para validar. [Documentação AWS+2Documentação AWS+2](#)